

Lab04 – Creating a Persistent Volume

In this exercise, you will create a PersistentVolume, connect it to a PersistentVolumeClaim and mount the claim to a specific path of a Pod.

1. Create a PersistentVolume named `pv`, access mode `ReadWriteMany`, 512Mi of storage capacity and the host path `/data/config`.

```
vagrant@kubernetes-control-plane:~/lab12$ vim pv.yaml
vagrant@kubernetes-control-plane:~/lab12$ cat pv.yaml
kind: PersistentVolume
apiVersion: v1
metadata:
  name: pv
spec:
  capacity:
    storage: 512Mi
  accessModes:
    - ReadWriteMany
  hostPath:
    path: /data/config
vagrant@kubernetes-control-plane:~/lab12$
vagrant@kubernetes-control-plane:~/lab12$ kubectl apply -f pv.yaml
persistentvolume/pv created
vagrant@kubernetes-control-plane:~/lab12$ kubectl get pv
NAME      CAPACITY   ACCESS MODES   RECLAIM POLICY   STATUS   CLAIM   STORAGECLASS   REASON   AGE
pv        512Mi      RWX            Retain           Available             12s
vagrant@kubernetes-control-plane:~/lab12$
vagrant@kubernetes-control-plane:~/lab12$
```

2. Create a PersistentVolumeClaim named `pvc`. The claim should request 256Mi and use an empty string value for the storage class. Ensure that the PersistentVolumeClaim is properly bound after its creation.

```
vagrant@kubernetes-control-plane:~/lab12$ vim pvc.yaml
vagrant@kubernetes-control-plane:~/lab12$ cat pvc.yaml
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: pvc
spec:
  accessModes:
    - ReadWriteMany
  storageClassName: ""
  resources:
    requests:
      storage: 256Mi
vagrant@kubernetes-control-plane:~/lab12$
vagrant@kubernetes-control-plane:~/lab12$ kubectl apply -f pvc.yaml
persistentvolumeclaim/pvc created
vagrant@kubernetes-control-plane:~/lab12$
vagrant@kubernetes-control-plane:~/lab12$ kubectl get pvc
NAME      STATUS   VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS   AGE
pvc       Bound    pv       512Mi      RWX              7s
vagrant@kubernetes-control-plane:~/lab12$
```

3. Mount the PersistentVolumeClaim from a new Pod named `app` with the path `/var/app/config`. The Pod uses the image `nginx:1.21.6`.

```
vagrant@kubernetes-control-plane:~/lab12$ vim pod.yaml
vagrant@kubernetes-control-plane:~/lab12$ cat pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: app
spec:
  containers:
  - image: nginx:1.21.6
    name: app
    volumeMounts:
    - mountPath: "/data/app/config"
      name: configpvc
  volumes:
  - name: configpvc
    persistentVolumeClaim:
      claimName: pvc
  restartPolicy: Never
vagrant@kubernetes-control-plane:~/lab12$
vagrant@kubernetes-control-plane:~/lab12$ kubectl apply -f pod.yaml
pod/app created
vagrant@kubernetes-control-plane:~/lab12$
vagrant@kubernetes-control-plane:~/lab12$
```

4. Open an interactive shell to the Pod and create a file in the directory `/var/app/config`.

```
vagrant@kubernetes-control-plane:~/lab12$ kubectl exec app -it -- /bin/sh
#
# cd /data/app/config
# ls
# ls -l
total 0
# touch test.txt
# ls -l
total 0
-rw-r--r-- 1 root root 0 Jul 20 17:16 test.txt
#
# exit
vagrant@kubernetes-control-plane:~/lab12$
```

5. Check that the `test.txt` file exists under the *hostPath* of the node.

```
brahim@Training:~/ckad-lab$ kubectl get pod -owide
NAME    READY   STATUS    RESTARTS   AGE   IP          NODE    NOMINATED NODE   READINESS GATES
app     1/1     Running   0           12m   10.44.0.1   kube-node1   <none>           <none>
brahim@Training:~/ckad-lab$
brahim@Training:~/ckad-lab$ vagrant ssh kube-node1
Last login: Sat Sep 16 15:03:08 2023 from 10.0.2.2
vagrant@kube-node1:~$
vagrant@kube-node1:~$ ls -l /data/config/
total 0
-rw-r--r-- 1 root root 0 Sep 16 15:01 test.txt
vagrant@kube-node1:~$
vagrant@kube-node1:~$ exit
logout
brahim@Training:~/ckad-lab$
```